

JavaScript Essentials

To make web pages interactive and provide dynamic functionality on the client side

Objectives

- Introduction to JavaScript
- JavaScript Variables & Operators
- Functions & Scope
- Errors
- Objects and Arrays
- DOM Manipulation
- Events

Introduction to JavaScript

- JavaScript is the **programming language** for the **Web**.
- To make web pages interactive and dynamic.
- Used for **front-end** & **server-side** development.
- The most common environment for JavaScript is within **web browsers**.
- JavaScript can also run on the server side using **Node.js**.
- **Dynamically typed**: No need to declare variable types.
- **Event-driven**: Responds to user interactions
- With Similarities to C-family languages.

Variables

- Variables are used to store data values temporarily.
- To Declare variables:

1. `var` (Legacy code): Function-scoped

```
var x = 10;
```

2. `let`: Block-scoped (works only within `{}`) and can be updated

```
let y = 20;
```

3. `const`: Block-scoped and cannot be reassigned.

```
const z = 30;
```

Dynamic Typing

- JavaScript uses **dynamic typing**
- **No need to declare** a variable's type
- **Determined at runtime** based on the assigned value
- A variable's type can **change during execution**
- Use *typeof* to check the current type of a variable

```
let myVariable; // No type declaration
myVariable = 42; // Now it's a number
myVariable = "Hello, World!"; // Now it's a string
myVariable = { key: "value" }; // Now it's an object

let num = 5;
let str = "10";
let result = num + str; // Result is "510" (string concatenation)
```

Datatypes

- **Primitive** & **Reference** data types.

Number: Represents numeric values, including integers and floating-point numbers.

String: Represents textual data, enclosed in single (') or double (") quotes.

Boolean: Represents true or false values. Example: **true** or **false**.

Undefined: Represents a variable that has been declared but not assigned a value.

Null: Represents the intentional absence of any object or value.

Object: Represents complex data structures and collections of key-value pairs.

Array: Represents ordered collections of values.

```
let description;  
let age = 22;  
let title = "Developer";
```

```
let person = { firstName: "Mehrdad", lastName: "Javan"};  
let numbers = [1, 2, 3, 4, 5];  
const pi = 3.14;
```

Operators

- Most operators in JS look and work the same

`+, -, %, =, *, /, ?, >, >=, ++`

- Checking equality in JS needs to be strict

`===` (checking equal value and equal type)

`==` (checking equal value)

`!=` (not equal value and not equal type)

Functions – syntax options

- To encapsulate and reuse blocks of code.
- `function functionName(param1, param2) { }`
- `Const functionName = function(param1, param2) { }`
- `Const functionName = (param1, param2) => { }`
- `() => ()`

```
function greet(name) {  
    return "Hello, " + name + "!";  
}  
let message = greet("JavaScript");  
console.log(message); // "Hello, JavaScript!"  
  
// Arrow functions provide a concise way to write functions.  
let multiply = (a, b) => a * b;
```

Function Scope

- Visibility and Accessibility of variables declared within a function.

Function Scope:

- Variables declared with `var` are function-scoped.
- They are accessible only within the function in which they are defined.

Block Scope:

- Variables declared with `let` and `const` are block-scoped.
- They are accessible within the block (a pair of curly braces) in which they are defined.
- If defined within a function, they are accessible only within that function.

Global Scope:

- Variables declared outside of any function or block have global scope.
- They can be accessed from any part of the code.

Errors

- Are known as exceptions or runtime errors.

- **Syntax Errors:**

Occur due to violations of JavaScript language rules.

Prevent code execution and typically involve missing or incorrect syntax elements.

- **Reference Errors:**

Happen when trying to access variables or objects that are not defined or out of scope.

Often result from mistyped variable names or accessing nonexistent object properties.

- **Type Errors:**

Arise when operations are performed on values of the wrong type.

Common examples include attempting to call non-functions as functions or accessing properties on undefined values.

- **Range Errors:**

Occur when accessing arrays or performing string operations with invalid indices or lengths.

Typically triggered when attempting to access elements beyond the array's boundaries.

- **Custom Errors:**

Can be created using the **Error** constructor to provide customized error messages and additional context for specific issues in the code.

```
throw new Error("This is a custom error message.");
```

Objects

- Objects are a fundamental concept in the language.
- JavaScript uses objects to represent data.
- Objects are like containers with keys and values.
- Keys are unique identifiers, and values can be anything.
- Objects are used to model complex data structures.

```
let person = {  
  id: 1,  
  firstName: "Mehrdad",  
  lastName: "Javan",  
  isTeacher: true  
};  
  
console.log(person.firstName); // Mehrdad  
console.log(person["id"]); // 1
```

```
let car = {  
  make: "Volvo",  
  model: "V60",  
  start: function() {  
    console.log("Engine started!");  
  },  
};  
car.start(); // Engine started!
```

Arrays

- Arrays are a fundamental data structure.
- To store collections of items or values.
- Elements in an array can be of different data types.
- Elements in an array are accessed by their index, starting from 0.
- JavaScript provides built-in methods for working with arrays(**push**, **pop**, **splice**, **concat** & etc.)
- Array iteration methods (**forEach**, **map**, **filter**, & **reduce**)

```
let fruits = ["apple", "banana", "cherry", "date"];
console.log(fruits[0]); // "apple"
console.log(fruits[2]); // "cherry"
fruits[1] = "grape"; // Modifies the second element
console.log(fruits.length); // 4
for (let i = 0; i < fruits.length; i++) {
  console.log(fruits[i]);
}
```

Use JavaScript in HTML

There are two ways to integrate **JavaScript** into **HTML** document

1. By writing code inside `<script>` tag

```
<div id="demo">Some content</div>
<script>
    document.getElementById("demo").innerHTML = "My First JavaScript";
</script>
```

2. By having the code in a separate file and link it to the page

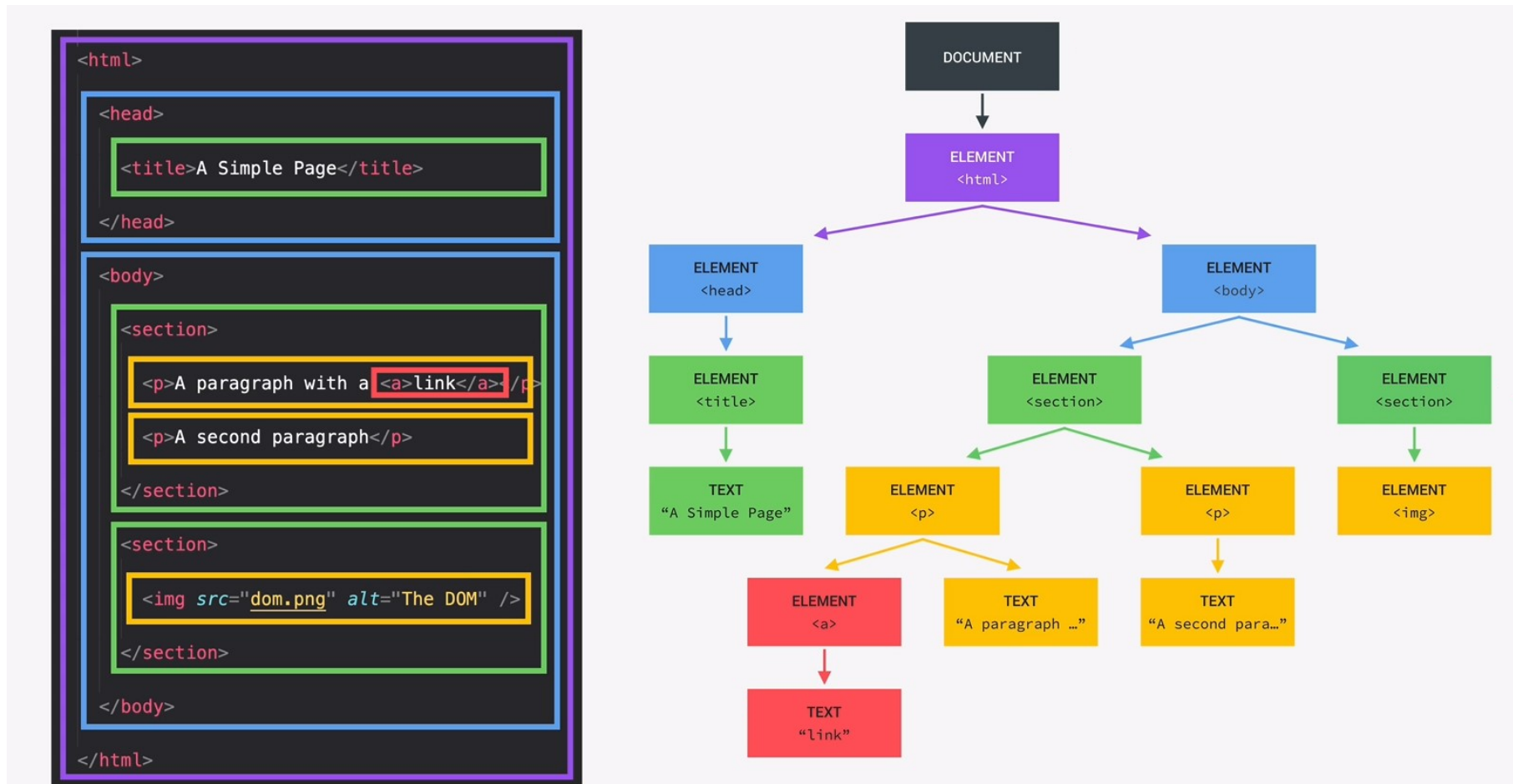
```
<div id="demo">Some content</div>
<script src="myjavascriptfile.js"></script>
```

Document Object Model (DOM)

- A programming interface (a toolkit) for **web documents**.
- Allows JavaScript to **interact with and manipulate** HTML and XML documents.
- Represents the **structure of a document** with a logical **tree**.
- Every **branch**, or every **element** in the DOM tree, is represented by a **node**.
- A node can represent various things like **elements**, **attributes**, or **text**.
- Every node in the DOM tree corresponds to a JavaScript **object**.
- Each object corresponds to an element in the document.
- DOM elements are organized in a parent-child relationship.
- Use DOM methods to modify the structure, or content of the document.

HTML DOM

- When a web page is loaded, the browser creates a **Document Object Model** of the page.



To access the HTML DOM Elements

Use methods and properties to interact with elements and make dynamic changes.

Finding HTML Elements

Method	Description
<code>document.getElementById(<i>id</i>)</code>	Find an element by element id
<code>document.getElementsByTagName(<i>name</i>)</code>	Find elements by tag name
<code>document.getElementsByClassName(<i>name</i>)</code>	Find elements by class name

```
const element = document.getElementById('myId');  
  
const elements = document.getElementsByClassName('myClass');  
  
const paragraphs = document.getElementsByTagName('p');  
  
const firstElement = document.querySelector('.myClass');
```

To access the HTML DOM Elements

Changing HTML Elements

Property	Description
<i>element.innerHTML = new html content</i>	Change the inner HTML of an element
<i>element.attribute = new value</i>	Change the attribute value of an HTML element
<i>element.style.property = new style</i>	Change the style of an HTML element
Method	Description
<i>element.setAttribute(attribute, value)</i>	Change the attribute value of an HTML element

```
const element = document.getElementById("main");
element.innerHTML = "new content";

element.style.color = 'blue';
```

To access the HTML DOM Elements

Adding and Deleting Elements

Method	Description
<code>document.createElement(<i>element</i>)</code>	Create an HTML element
<code>document.removeChild(<i>element</i>)</code>	Remove an HTML element
<code>document.appendChild(<i>element</i>)</code>	Add an HTML element
<code>document.replaceChild(<i>new</i>, <i>old</i>)</code>	Replace an HTML element
<code>document.write(<i>text</i>)</code>	Write into the HTML output stream

To create a table element

```
// Create a table element
const table = document.createElement('table');
table.setAttribute('id', 'myTable');

// Create table header (th) row
const headerRow = table.insertRow();
const headerCell1 = headerRow.insertCell(0);
const headerCell2 = headerRow.insertCell(1);
headerCell1.textContent = 'Name';
headerCell2.textContent = 'Age';

// Create table rows with data
const dataRows = [ ['John Doe', '25'], ['Jane Smith', '30'] ];

dataRows.forEach(data => {
  const row = table.insertRow();
  data.forEach((cellData, index) => {
    const cell = row.insertCell(index);
    cell.textContent = cellData;
  });
});

// Add the table to the body of the HTML document
document.body.appendChild(table);
```

DOM Events

- In JavaScript, events are actions that happen in the browser.
- JavaScript makes web pages interactive by catching and responding to events.
- Examples of HTML events:
 - Click a button
 - Type on the keyboard
 - Resize the window
 - Move the mouse over an element
 - Change something in an input field

Enter Email

Click Me

Email Is not Valid

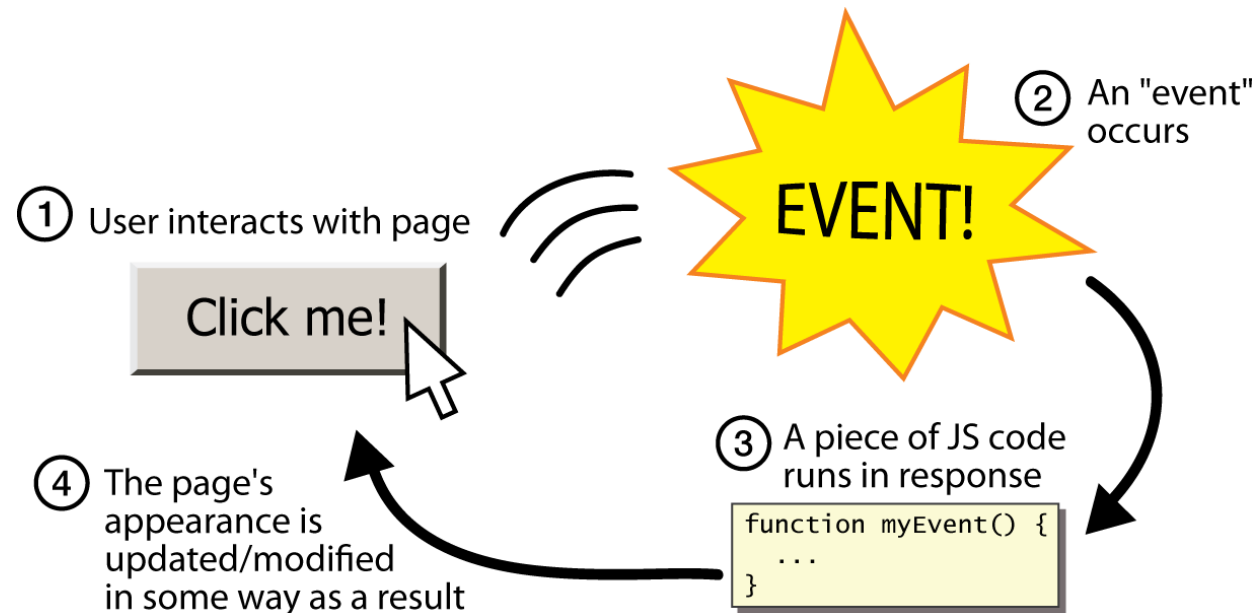
DOM Event Listener

- Attaches functions to respond to specific events on HTML elements.
- Use `addEventListener` method to add an event listener to an element.
- Use `removeEventListener` method to remove an event listener on an element.

```
element.addEventListener("click", myFunction );
```

What is an Event ?

JavaScript's interaction with **HTML** is handled through **events** that occur when the user or the browser manipulates a page. When the page loads, it is called an **event**. When the user clicks a button, that click too is an **event**. Other examples include **events** like pressing any key, closing a window, resizing a window, etc.



Reference

- https://www.tutorialspoint.com/javascript/javascript_overview.htm
- <https://www.w3schools.com/>
- [https://developer.mozilla.org/en-US/docs/Web/API/Document Object Model/Introduction](https://developer.mozilla.org/en-US/docs/Web/API/Document_Object_Model/Introduction)